

LUMEN · 2026

Lumen — Hybrid Search on the Lakehouse

Semantic + full-text product search, fused inside Postgres (Lakebase)

Lakebase / pgvector

BGE-large + RRF

Technical overview

Goal: understand how Lumen delivers high-quality product search in a single Postgres — and how it compares to dedicated search engines and vector databases.

Rafael Arana · Senior Solutions Architect

May 29, 2026

Agenda

1. **Introduction** — what Lumen is
2. **Advantages** of Lumen vs traditional search engines
3. **General architecture**
4. **Detailed architecture**
5. **How search works** — semantic, full-text, hybrid (RRF) + how we measure quality & latency
6. **Comparison** — Elasticsearch / OpenSearch & Vector DBs vs Lumen
7. **Resources & references**

1 · Introduction

"One Postgres for data, vectors and keywords — no separate search cluster to run."

What is Lumen?

- An **e-commerce product search** reference app over the **WANDS** catalog (~43K products).
- Built entirely on Databricks: **Lakehouse** for data + embeddings, **Lakebase** (managed Postgres) for serving, a **Databricks App** (FastAPI + React) for the UI.
- Two retrieval modes, chosen per request:
 - **Semantic** — vector similarity over BGE-large embeddings.
 - **Hybrid** — semantic + full-text, fused with Reciprocal Rank Fusion.

Everything — data, vector index, full-text index, fusion logic — lives in **one database**. Search is a SQL function call.

2 · Advantages vs traditional search engines

Fewer moving parts, governed, and good at both meaning and keywords.

Why Lumen, vs a dedicated engine

Operational simplicity

- **One system**, not two — no separate search cluster.
- **No sync pipeline** / dual source of truth.
- Transactional consistency with the operational data.

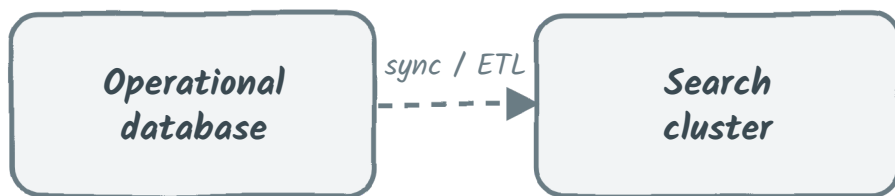
Capability & governance

- **Semantic + keyword** together, fused with RRF.
- Governed by **Unity Catalog**; least-privilege grants.
- Low latency via in-process caching (p99 $\approx$ 50 ms).

The trade: you give up a distributed engine's massive horizontal scale and BM25-grade text ranking — in exchange for one governed system with no synchronization. Great fit when search complements, rather than is, the product.

One database, not two

Traditional



2 systems · dual source of truth · sync lag & ops

Lumen



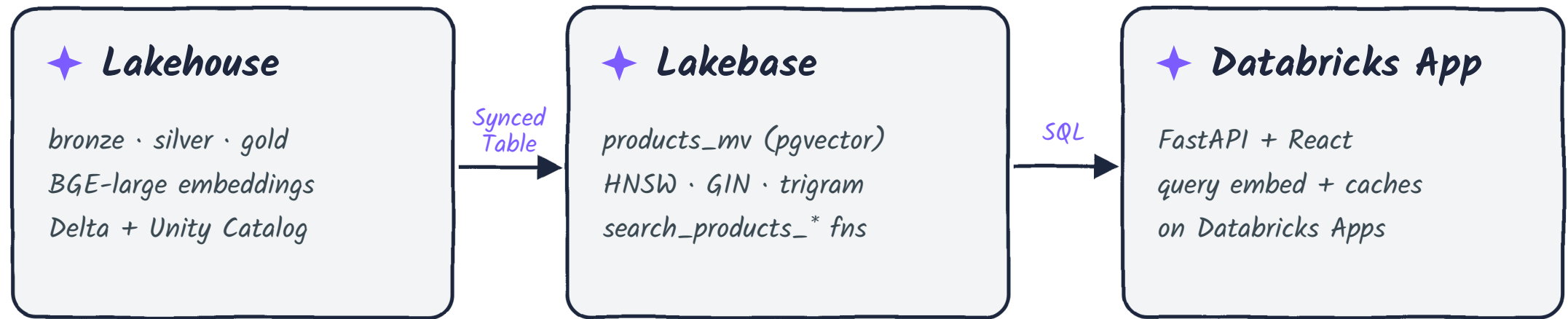
1 engine · no sync · Unity Catalog governance

No separate cluster, no pipeline to keep in sync — search is a **SQL function** next to the data.

3 · General architecture

Lakehouse builds it · Lakebase serves it · the App exposes it.

The three planes



Lakehouse **builds** the index · Lakebase **serves** it · the App **exposes** it.

4 · Detailed architecture

What actually lives in Lakebase.

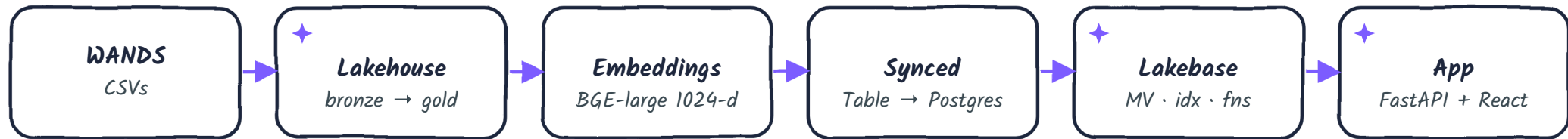
Serving objects in Lakebase (Postgres)

Object	Purpose
<code>products_mv</code> (materialized view)	casts the synced <code>jsonb</code> embedding to a real <code>vector(1024)</code> ; holds <code>search_vector</code> (tsvector)
<code>idx..._embedding</code> — HNSW	approximate nearest-neighbor vector search (<code>m=16</code> , <code>ef_construction=200</code> , cosine)
<code>idx..._fts</code> — GIN	full-text matching on <code>search_vector</code>
<code>idx..._name_trgm</code> — GIN (pg_trgm)	trigram fuzzy match on the product name (typo fallback)
<code>search_products_semantic / _hybrid</code>	PL/pgSQL serving functions (vector-only / RRF hybrid)

The App backend adds three cache layers (result, embedding, full path) — Turbo mode — affecting latency only, never ranking.



Data flow, end to end



Build-time pipeline (left → middle) runs in the Lakehouse; the **Synced Table** hands off to Lakebase, which the App queries at request time.

5 · How search works

Two signals, fused by rank — not by score.

Two signals

Semantic (vector)

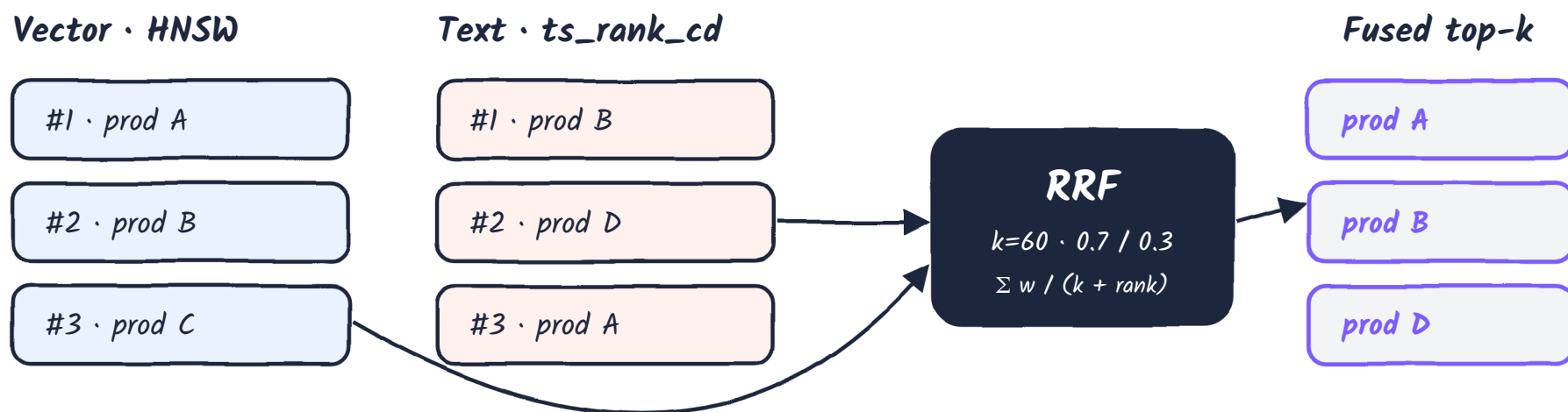
- Query → 1024-dim BGE-large embedding.
- HNSW cosine search (`<=>`).
- Captures **intent & synonyms**.

Full-text (keyword)

- `tsvector` + GIN, `plainto_tsquery`.
- **Name-weighted** ranking via `setweight` + `ts_rank_cd`.
- **pg_trgm** fuzzy fallback for typos.

Postgres FTS has no true BM25/IDF — `setweight` + `ts_rank_cd` approximate it with field boosting + proximity.

How RRF fuses the two lists



Ranks are fused, not raw scores — a product strong in **either** list rises; strong in **both** wins.

Hybrid fusion — Reciprocal Rank Fusion

```
WITH vector_results AS ( -- top 3*k by cosine distance, ROW_NUMBER() = rk
... ORDER BY embedding <=> query_embedding ),
text_results AS ( -- top 3*k by name-weighted ts_rank_cd, ROW_NUMBER() = rk
... ORDER BY ts_rank_cd('{0.1,0.3,0.6,1.0}', weighted_tsv, query) DESC )
SELECT product_id,
       0.7 * (1.0 / (60 + COALESCE(v.rk, 1000))) -- vector weight
+ 0.3 * (1.0 / (60 + COALESCE(t.rk, 1000))) -- text weight
       AS rrf_score
FROM vector_results v FULL OUTER JOIN text_results t USING (product_id)
ORDER BY rrf_score DESC LIMIT k;
```

Ranks are fused (not raw scores) because cosine distance and `ts_rank_cd` are not comparable. `k=60` damping; missing rank → 1000.

How we measure performance

Search quality — "are results good?"

- Ground truth: **WANDS graded labels** (Exact/Partial/Irrelevant).
- Metrics: **NDCG@k**, Recall@k, Precision@k, MRR, MAP.
- Harness in `eval/` calls the SQL functions **directly** — no app, no cache → pure ranking.
- Two query sets: **clean** + **seeded-typo**; reproducible (fixed seed & `ef_search`).

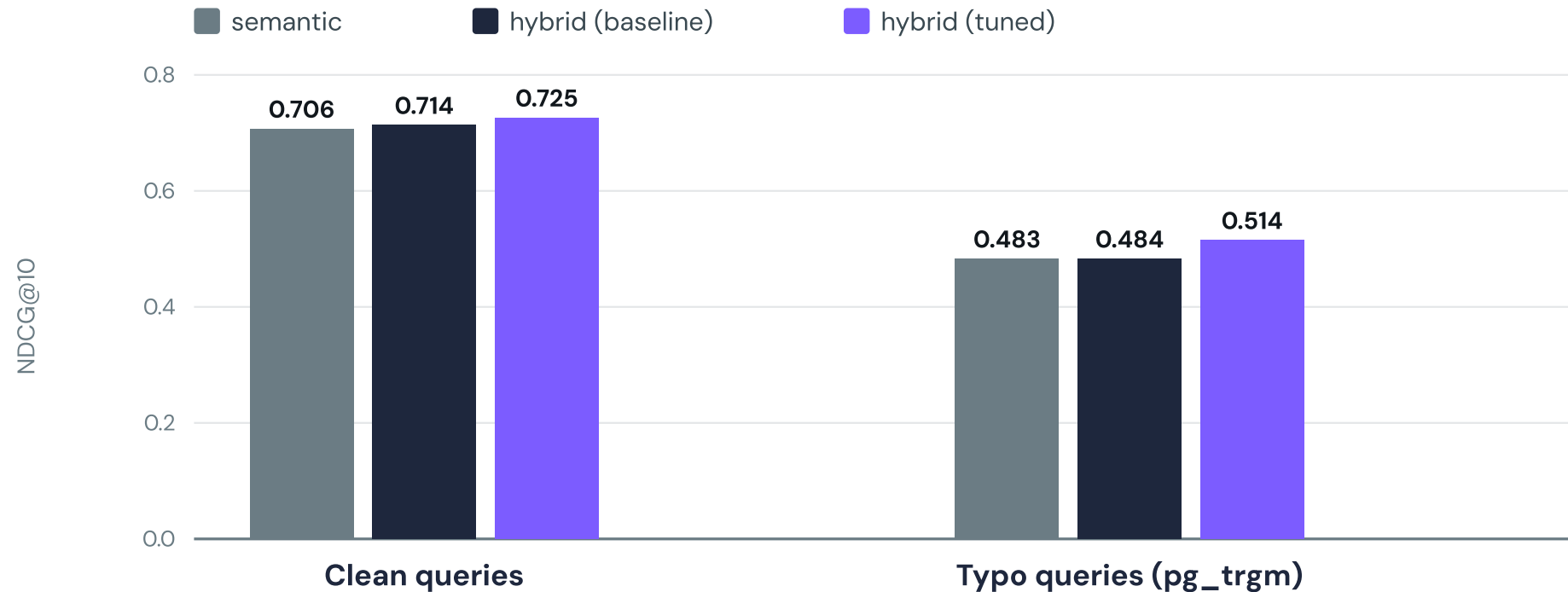
Latency — "is it fast?"

- In-app **load generator**: async workers, 4 buckets (standard/turbo × semantic/hybrid).
- Reports **p50 / p95 / p99**, req/s, error count per bucket.
- External **Goose** load test adds Apps edge auth + WAN RTT.
- Measured at **each optimization stage** to attribute the gain.

Quality and speed are measured **separately and repeatably** — so every change is judged on both, not on intuition.

Quality, measured against WANDS labels

Graded judgments (Exact/Partial/Irrelevant) → NDCG, Recall, MRR. Hybrid, before vs after FTS tuning:



Benefit: name-weighting lifts hybrid on clean queries with no regression, and the `pg_trgm` fallback makes hybrid clearly beat semantic under typos — where before it added nothing.

Measuring latency — methodology

In-app benchmark (backend-only)

- `POST /api/benchmark` spins async workers against **localhost** — no edge-auth noise.
- Independently tracks **4 buckets**: standard/turbo × semantic/hybrid.
- Reports p50/p75/p95/p99/max, req/s, errors.

Goose load test (end-to-end)

- Rust/Goose from a laptop → includes **Apps edge auth + WAN RTT** (~+60–100 ms).
- Ramps users/concurrency to check **scaling** & error rate.

Same 100 seed queries in both; backend-only numbers isolate search cost, end-to-end numbers show what users feel.

Latency results & benefits

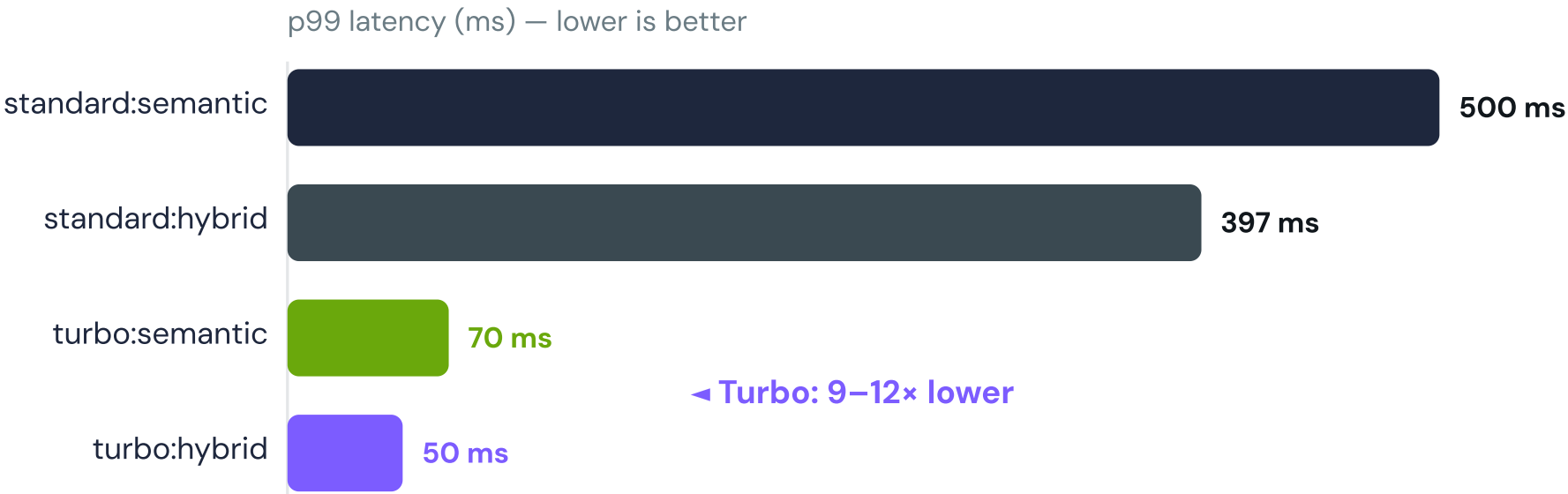
In-app benchmark · 10 workers × 30 s · 50/50 Standard/Turbo · all optimizations on:

Bucket	p50	p95	p99	req/s
standard:semantic	84 ms	127 ms	500 ms	59.0
standard:hybrid	84 ms	123 ms	397 ms	25.3
turbo:semantic	19 ms	36 ms	70 ms	60.2
turbo:hybrid	18 ms	35 ms	50 ms	24.9
Aggregate	62 ms	106 ms	275 ms	169.4

Benefit: the 3-layer cache + warm pool + `ef_search=20` cut Turbo **p99 from 622 ms → 50–70 ms (9–12×)** and p50 91 → 18 ms (5×). **0 errors** across 5,208 requests; Goose shows throughput nearly doubling at 2.5× concurrency — it scales cleanly.



Standard vs Turbo — p99 latency



The cache + warm pool collapse tail latency: Turbo's p99 (50–70 ms) is mostly network & serialization, not search.

6 · Comparison

Elasticsearch / OpenSearch & Vector DBs vs Lumen.

Lumen vs Elasticsearch / OpenSearch

Dimension	Elasticsearch / OpenSearch	Lumen (Postgres FTS + pgvector)
Text ranking	BM25 (TF-IDF + length norm)	<code>ts_rank_cd</code> + field weighting (no true IDF)
Text features	fuzzy, synonyms, n-grams, highlighting, facets	stemming + <code>pg_trgm</code> fuzzy fallback
Scale	distributed, sharded, billions of docs	single-node (+ read replicas), 10^4 – 10^7 docs
Hybrid + vectors	native (kNN + RRF retrievers, 8.x)	RRF written in SQL; pgvector HNSW
Operations	separate cluster + sync pipeline	one Postgres; no sync; UC-governed

Same idea (inverted index + ranking + vectors); ES is the dedicated, distributed engine with better text ranking, Lumen folds it into the database.



Dedicated Vector DB vs Lumen

Dimension	Dedicated Vector DB (Pinecone, Weaviate, Milvus...)	Lumen (pgvector in Lakebase)
Vector search	purpose-built ANN, very large scale	HNSW via pgvector, ample for this corpus
Keyword / hybrid	often add-on or external	full-text + RRF in the same engine
Joins & filters	limited; metadata filters only	full SQL: joins, facets, transactional rows
Data movement	separate store to populate & sync	none — vectors sit next to the data
Governance	separate IAM	Unity Catalog + Postgres roles

When a dedicated engine wins: billions of vectors / very high QPS, or search-grade text features (BM25, synonyms, typo-tolerance everywhere) are the core product. Otherwise, keeping it in the database removes a whole system.



7 · Resources & references

| Where to dig deeper.

Resources & references

Topic	Link
Lumen — architecture	doc/Architecture.md
Lumen — search deep-dive	doc/Search.md
Lumen — eval harness	eval/README.md
WANDS dataset	github.com/wayfair/WANDS
pgvector	github.com/pgvector/pgvector
PostgreSQL full-text search	postgresql.org/docs/current/textsearch.html
pg_trgm	postgresql.org/docs/current/pgtrgm.html
Reciprocal Rank Fusion (Cormack et al., 2009)	cormacksigir09-rrf.pdf
BGE-large embeddings	huggingface.co/BAAI/bge-large-en-v1.5
Databricks Lakebase	databricks.com/product/lakebase



Thank you

Rafael Arana

Senior Solutions Architect

rafael.arana@databricks.com

Questions welcome.